Chapter 1

Embedded Digital Communications A

ENEL712 Embedded Systems Design — Lecture 5

Summary

Introduction to embedded communications: parallel vs serial, asynchronous (RS-232/422/485), synchronous (SPI, I^2C), and other notable protocols (CAN, PS/2).

Topics Covered

- Introduction to embedded communication
- Parallel vs serial communication
- Asynchronous serial protocols
- Synchronous serial protocols
- Other notable protocols

1.1 Introduction to Embedded Communication

Embedded systems frequently need MCUs to talk to other ICs, sensors, or PCs, transferring multiple bytes rather than single bits. Hundreds of protocols exist, but ~6 are common across most MCUs.

1.2 Parallel vs. Serial Communication

- Parallel: multiple bits transmitted concurrently across a bus.
- Serial: one bit at a time over a single wire.

1.2.1 Legacy parallel interfaces

- PC Parallel Port (DB-25): 8 data pins + control lines; superseded by USB and Ethernet.
- IDE (Parallel ATA): 40-pin cable, 16 parallel data bits; superseded by SATA.

- LCD parallel: 4-bit or 8-bit data with 3 control lines and timing requirements.

1.3 Asynchronous Serial (RS-232 / RS-422 / RS-485)

- TX and RX have separate clocks and must agree on a baud rate.
- Frames synchronised by start and stop bits.

1.3.1 RS-232

- Three wires for full-duplex (Tx, Rx, GND).
- True RS-232 uses ±15 V; TTL RS-232 (common in MCUs) uses 0–3.3 V or 0–5 V.
- Optional parity bit for single-bit error detection.
- Unbalanced (single-ended) signalling — sensitive to noise; short distances/lower rates.

1.3.2 RS-422 / RS-485

- Differential signalling for noise immunity, range >1 km, and higher data rates.
- RS-485 is multi-point — up to 32 transceivers on a single bus.

1.4 Synchronous Serial (SPI, I^2C)

1.4.1 SPI (Serial Peripheral Interface)

- Four wires: SCK, MOSI, MISO, SS.
- Full-duplex; master generates clock and initiates all transfers.
- Highly flexible/fast but interpretation is software-dependent.
- Four modes from CPOL/CPHA combinations (Mode 0–3) define when the master clocks and when data is sampled.

1.4.2 I^2C (Inter-Integrated Circuit)

- Two wires: SCL and SDA.
- Multi-master with up to 1008 slaves via 7-bit or 10-bit addressing.
- Open-collector ports require pull-up resistors to define the high state.
- Half-duplex; mandatory ACK bit after every 8 bits ensures reliability.

1.5 Other Notable Protocols

- CAN (Controller Area Network): asynchronous, message-based, fault-tolerant; used in automotive/industrial. Differential signalling and non-destructive bitwise arbitration by message ID priority.
- PS/2: synchronous protocol similar to I^2C; older keyboards/mice.

1.6 Use Cases & When to Pick What

Protocol	Pick when...	Typical example
UART/RS-232	Two devices, modest rate, simple framing	MCU ↔ PC debug terminal, GPS module
RS-485	Long bus, many drops, noisy environment	Industrial sensors on a shop floor
SPI	One master, fast point-to-point, full-duplex	SD card, OLED display, ADC
I²C	Many small sensors on the same two pins	Temperature, humidity, EEPROM combo
CAN	Distributed control; messages, not addressing; tolerant of faults	Automotive ECUs, factory automation
PS/2	Legacy keyboard/mouse interfaces	Retro computing

1.7 Formulas & Calculations

1.7.1 Effective vs. raw UART byte rate

Each byte costs 10 bits on the wire (8 data + start + stop). At baud rate B bps, effective byte rate ≈ B / 10. So a 115,200 baud link gives ≈ 11,520 bytes per second.

1.7.2 I²C ACK timing

An I²C transaction is at least 1 (start) + 7 (addr) + 1 (R/W) + 1 (ACK) + 8 (data) + 1 (ACK) + 1 (stop) = 20 bit periods. At 100 kHz that is 200 μs per byte payload — slower than SPI but acceptable for sub-Hz sensors.

1.8 Worked Example: SPI Mode Picking

An SD card datasheet calls out SPI Mode 0 (CPOL = 0, CPHA = 0). That means the clock idles low and data is sampled on the rising edge. Setting your SPI peripheral to Mode 3 instead (CPOL = 1, CPHA = 1) is a classic 'works in the lab, dies in the field' bug: the card will respond to some commands but garble multi-byte transfers because the sample point is shifted by half a bit.

1.9 Code Snippets

1.9.1 AVR UART transmit one byte

```
void uart_putc(uint8_t c) {
    while (!(UCSR0A & (1<<UDRE0))) ;  // wait until tx buffer is empty
    UDR0 = c;                          // write triggers transmission
}
```

1.9.2 Reading an I²C sensor

```
// 7-bit address 0x48, register 0x00
uint8_t value;
i2c_start();
i2c_write((0x48 << 1) | 0); // address + write bit
```

```
i2c_write(0x00);            // register pointer
i2c_repeated_start();
i2c_write((0x48 << 1) | 1); // address + read bit
value = i2c_read(false);    // NACK = stop after this byte
i2c_stop();
```

Key Definitions

Baud Rate
Rate at which symbols (bits/signals) are transmitted per second on a serial link.

Full-Duplex
Channel that carries simultaneous, two-way traffic (e.g., SPI).

Half-Duplex
Channel that allows two-way comms one direction at a time (e.g., I^2C, RS-485).

Parity Bit
Optional bit added to detect single-bit errors.

Differential Signalling
Two complementary lines whose difference carries the signal; rejects common-mode noise (RS-422/485, CAN).

Open-Collector
Output that pulls low or floats; needs a pull-up to define the high state (e.g., I^2C).

Arbitration
Bus access process used in multi-master systems (like CAN) based on message priority.

Bit Period
Duration of one bit at the current baud/clock rate; 1 / B seconds.

Repeated Start
I^2C primitive that issues a new START condition without an intervening STOP — used to switch from write to read on the same device.

Frame Overhead
Bits added to each character for synchronisation/error detection (e.g., start, parity, stop) that lower effective throughput.

Sources

- Lecture 5.pdf